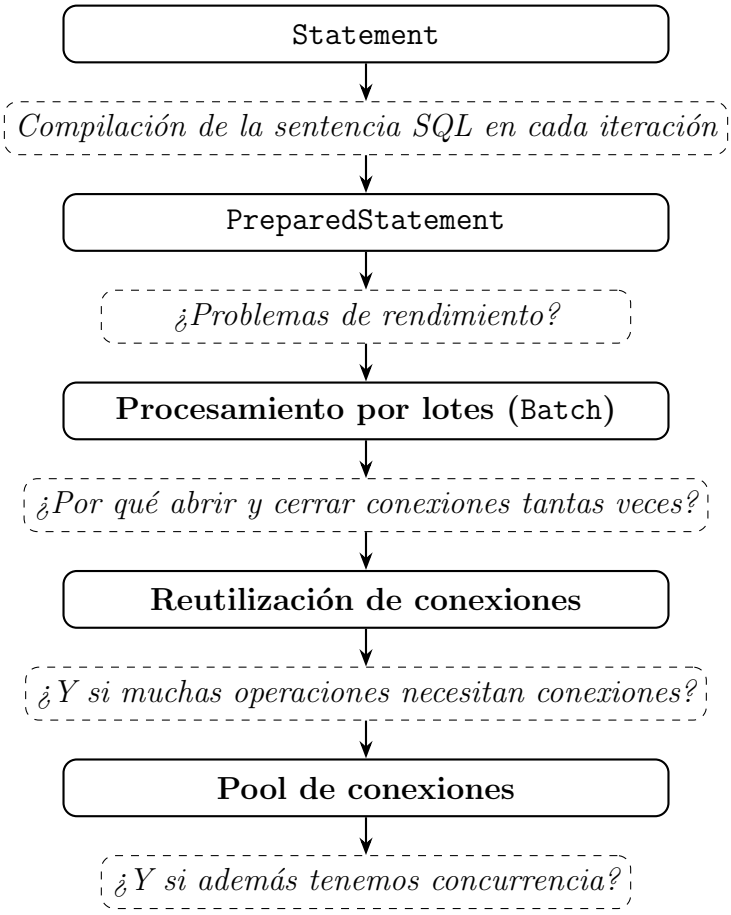
Práctica 1: Costes de acceso a una base de datos

1 Objetivos

- Conexiones a diversos SGBD: Oracle, HSQLDB.
- Comparativa `Statement`/`PreparedStatement`.
- Uso de un Pool de conexiones y comparativa de tiempos.

La secuencia lógica de la práctica será la siguiente:



2 Preparación

1. Descarga el material del Campus Virtual (base de datos HSQLDB, script Oracle, drivers).
2. Descomprimir el fichero cws_database y arrancar el servidor HSQLDB (ejecutar `1.startup`).
3. Conectarse a Oracle (ej. SQLDeveloper). Los datos de conexión están en el apartado 3.
4. Crear un directorio workspace y arrancar Eclipse.

2.1 Construir la base de datos Oracle

Una vez conectado a Oracle, es necesario eliminar las tablas de la base de datos y crear las nuevas.

a. Si ya has cursado la asignatura en años anteriores, en primer lugar debes eliminar de la base de datos las tablas de años anteriores. Para ello ejecuta el comando:

```
SELECT 'DROP TABLE '||table_name||' CASCADE CONSTRAINTS;'
FROM user_tables
WHERE table_name LIKE 'T%'
```

b. El resultado es la lista de comandos de borrado que debes ejecutar. Cópialo y ejecútalo.

c. Ejecuta el script `script_create_carworkshop_Oracle.sql` para crear las nuevas tablas.

2.2 Crear y configurar un proyecto Eclipse

- Crea un proyecto en Eclipse para escribir los ejercicios.
- Configura el proyecto
  - Crea en el proyecto una carpeta (no código) `lib`. Copia ahí los ficheros `.jar` de los dos drivers (Oracle, HSQLDB).
  - Configura el build path de la aplicación para hacer referencia a ambos (Add external jar).

3 Datos de conexión

Parámetro	**Oracle**	**HSQLDB**
DRIVER	`thin`	`hsql`
HOST / IP	`156.35.94.98`	`localhost`
PUERTO	`1521`	–
SID (id)	`desa19`	–
USER	`UO`	`sa`
PASS	`password Oracle`	`""`

4 Ejercicios

4.1 Comparar el rendimiento de Statement vs. PreparedStatement

Crea una clase llamada `StatementBenchmark` para el primer ejercicio.

El programa debe conectarse a la base de datos y consultar unos datos. El objetivo es medir el tiempo empleado en una tarea, realizándola de dos maneras: utilizando *Statement*, y *PreparedStatement*.

Para que la diferencia de tiempo de ejecución sea significativa, la sentencia debe ejecutarse suficientes veces, por ejemplo **1000**, **cambiando ligeramente** cada vez. En la base de datos HSQLDB puede ejecutarse incluso 10000; en Oracle supondría mucho tiempo de ejecución, esperando la salida.

Se sugiere la sentencia: **Obtener de la tabla TINVOICES, el identificador de una factura que tiene una cantidad a pagar menor que el valor de la variable de control del bucle**.

Para medir tiempos puedes utilizar `System.currentTimeMillis()`, que devuelve la fecha/hora en milisegundos desde el 01/01/1970.

4.1.1 ¿Es más eficiente utilizar PreparedStatement que Statement?

Para comprobarlo, ejecuta varias veces cada versión y toma nota de los resultados.

Tiempos de ejecución

Tiempo (ms)	HSQLDB		ORACLE	
	Statement	PreparedStatement	Statement	PreparedStatement
run 1				
run 2				
run 3				

¿Se observa una diferencia significativa entre Statement y PreparedStatement en HSQLDB? ¿Y en Oracle? ¿Qué factores pueden explicar los resultados?

4.2 Batch processing

En el ejercicio anterior hemos ejecutado las operaciones de una en una.

Cuando la base de datos se encuentra en un servidor remoto (como es el caso de Oracle), cada operación puede implicar una comunicación entre el programa cliente y el servidor de base de datos.

¿Qué ocurre cuando tenemos que ejecutar muchas operaciones? ¿Podemos reducir el coste de estas comunicaciones agrupando las operaciones en un lote?

JDBC proporciona el mecanismo de **batch processing**, que permite agrupar varias operaciones en un lote, reduciendo el número de interacciones con el servidor.

Así, si una parte importante del coste de la operación se debe a la comunicación con el servidor, agrupar varias operaciones en un lote debería reducir significativamente el tiempo de ejecución.

El propósito de este ejercicio es medir y comparar el impacto de ejecutar varias sentencias sql en un lote frente a hacerlo de una en una.

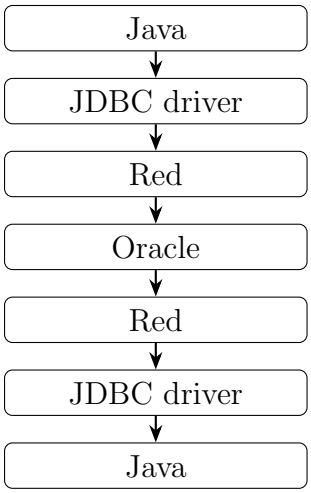
Crea una clase llamada `BatchBenchmark`. Debe conectarse a la base de datos Oracle y actualizar (no se puede ejecutar consultas en batch) varias filas utilizando dos estrategias:

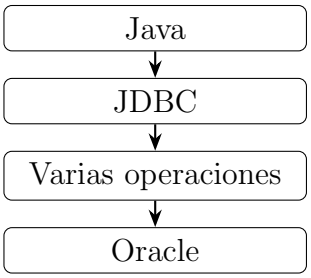
- ejecutando las operaciones individualmente;
- agrupándolas mediante *JDBC batch processing*. Puedes encontrar un ejemplo aquí: `https://www.baeldung.com/jdbc-batch-processing`

Se sugiere la sentencia **Actualizar la tabla TINVOICES, añadiendo al campo amount el valor de la variable de control del bucle cuando el estado sea PAID**.

1. **Sin lote**
   - Crear la conexión
   - Crear la prepared statement
   - Ejecutar un bucle. En cada iteración:
     - Completar la sentencia a ejecutar.
     - Ejecutarla.
2. **Con lote**

- Crear la conexión
- Crear la prepared statement
- Ejecutar un bucle. En cada iteración:
  - Completar la sentencia a ejecutar.
  - Añadirla al lote (`addBatch`).





Para comprobar la posible mejora de la ejecución por lotes, ejecuta varias veces cada escenario y toma nota de los resultados.

Tiempos de ejecución

Tiempo (ms)	ORACLE	
	sin lote	con lote
run 1		
run 2		
run 3		

Se puede ver una discusión muy interesante al respecto de lo trabajado en este ejercicio aquí.

4.3 Comprobar el coste de apertura y cierre de conexiones

Una conexión a base de datos es un recurso costoso, tanto en tiempo como en memoria, ya que gestiona todas las comunicaciones entre la aplicación cliente y el servidor.

Crea una clase llamada `ConnectionBenchmark` para el segundo ejercicio.

El propósito de este ejercicio es medir y comparar el impacto de abrir y cerrar conexiones frente a reutilizarlas, desarrollando un programa que evalúe los tiempos de ejecución en dos escenarios distintos:

La sentencia a ejecutar será la misma que en el ejercicio anterior y utilizaremos en ambos casos PreparedStatement.

1. **Conexión dentro del bucle**

   - Ejecutar el bucle. En cada iteración:
     - Crear una conexión.
     - Completar la sentencia.

  - Ejecutar la sentencia.
  - Cerrar la conexión.

2. **Conexión fuera del bucle**

   - Crear la conexión antes de iniciar el bucle.
   - Ejecutar el bucle. En cada iteración:
     - Completar la sentencia.
     - Ejecutar la sentencia.
   - Cerrar la conexión al finalizar el bucle.

La comparación de ambos escenarios permitirá observar la diferencia de coste entre establecer una conexión repetidamente o reutilizar una conexión abierta durante varias operaciones.

Para comprobarlo, ejecuta varias veces cada escenario y toma nota de los resultados.

Tiempos de ejecución

Tiempo (ms)	HSQLDB		ORACLE	
	in loop	out loop	in loop	out loop
run 1				
run 2				
run 3				

4.4 Comparar el coste de establecer una conexión física con el SGBD, frente a reutilizar una conexión existente de un pool de conexiones

El objetivo de este ejercicio es **comparar el coste de crear conexiones físicas con la base de datos frente a la utilización de un pool de conexiones con la misma**.

Utilizaremos el pool de conexiones c3p0, cuyos drivers se encuentran en la carpeta `lib` (descargados desde `https://www.mchange.com/projects/c3p0/`).

Crea una clase llamada `ConnectionPoolBenchmark` para este ejercicio.

En ella se accederá varias veces a una base de datos y en cada ocasión se realizará una operación. Como en los demás ejercicios, se implementarán dos escenarios distintos en los que, para cada operación con la base de datos:

- **Sin pool de conexiones**: se crea y cierra su propia conexión.
- **Con pool de conexiones**: se obtiene una conexión lógica del pool y al finalizar se devuelve al pool.

El propósito es **medir los tiempos de ejecución** en ambos casos y analizar cómo el uso de un pool de conexiones mejora la eficiencia y el rendimiento del acceso a la base de datos.

Ejemplo de bucle sin pool

```
for (int i = 0; i < 1000; i++) {
 crearConexion();
 ejecutar sentencia();
 cerrarConexion();
}
```

Ejemplo de bucle con pool

```
crearPoolConexiones();
for (int i = 0; i < 1000; i++) {
 getConexionFromPool();
 ejecutar sentencia();
 releaseConexion();
}
```

En el segundo escenario, a las instrucciones de conexión y acceso a la base de datos, se le añade la creación y configuración de un pool de conexiones con las siguientes características (valores por defecto de la librería):

- Máximo de conexiones (`maxPoolSize`): 15.
- Mínimo de conexiones (`minPoolSize`): 3.
- Tamaño inicial del pool (`initialPoolSize`): 3.

4.4.1 Ejemplo de inicialización de c3p0

A modo de referencia, se puede usar el siguiente extracto de código disponible en la web de c3p0 para crear un `PooledDataSource`.

```
DataSource ds_unpooled = DataSources.unpooledDataSource(URL, USER, PASS);

Map<String, Object> overrides = new HashMap<>();

/* Las siguientes tres instrucciones se pueden omitir
   si los valores que se usan son los valores por defecto */
overrides.put("minPoolSize", 3);
overrides.put("maxPoolSize", 15);
overrides.put("initialPoolSize", 3);

DataSource ds_pooled =
    DataSources.pooledDataSource(ds_unpooled, overrides);

// Para obtener conexion del pool
Connection con = ds_pooled.getConnection();
```

4.4.2 Ejecución repetida de cada escenario

Ejecuta varias veces cada escenario y toma nota de lo siguiente.

Tiempos de ejecución

	HSQLDB	ORACLE
Sin pool de conexiones		
Con pool de conexiones (Medición A)		
Con pool de conexiones (Medición B)		

Medición A — Incluyendo la creación del pool Toma los tiempos **incluyendo la creación del pool** de conexiones. Esto sirve para responder a la pregunta: ¿Compensa utilizar un pool en una ejecución pequeña?

Medición A — Pool ya creado Toma el tiempo **después de crear el pool**, justo antes de iniciar el bucle. Con esto respondemos a la pregunta: ¿Cuál es el coste de utilizar un pool una vez que ya está inicializado?

Si la tarea fuese suficientemente costosa, sería todavía mejor dividirla en subtareas (Threads), cada uno de los cuales ejecutaría parte de las mismas.